Project 5 - Messy Dataset Cleaning and Preparation for Analysis

Take a **raw messy dataset** and transform it into a **clean dataset ready for analysis**.

You will fix:

Problem	Example
Missing values	NULL salary
Wrong data types	Age stored as text
Duplicates	Same customer appearing twice
Inconsistent values	"Male", "male", "M"
Extra spaces	" Delhi "
Wrong formats	Date stored as text

Note:- Download 'Netflix Movies and TV Shows' dataset from Kaggle as:-

Problems inside dataset:

- ✓ Missing directors
- ✓ Missing country
- ✓ Wrong date format
- ✓ Multiple countries in one column
- ✓ Duplicates

Perfect for practice.

Step 1 — Explore Dataset

```
import pandas as pd
import numpy as np

df = pd.read_csv("netflix_titles.csv")
df.head()
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	descript
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As father ne the enc his film
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	A cross paths part Cape To
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act	To pro his far fro powe drug

```
df.shape
```

Output:- (8807, 12)

```
df.columns
```

Output:- Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
'release_year', 'rating', 'duration', 'listed_in', 'description'],
dtype='str')

df.describe()

	release_year
count	8807.000000
mean	2014.180198
std	8.819312
min	1925.000000
25%	2013.000000
50%	2017.000000
75%	2019.000000
max	2021.000000

df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   str
 1   type          8807 non-null   str
 2   title         8807 non-null   str
 3   director      6173 non-null   str
 4   cast          7982 non-null   str
 5   country       7976 non-null   str
 6   date_added    8797 non-null   str
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   str
 9   duration      8804 non-null   str
 10  listed_in     8807 non-null   str
 11  description   8807 non-null   str
dtypes: int64(1), str(11)
memory usage: 825.8 KB
```

df.isnull().sum()

Output:

show_id	0
type	0
title	0
director	2634
cast	825
country	831
date_added	10
release_year	0
rating	4
duration	3
listed_in	0
description	0

dtype: int64

Step – 2 Handle Missing Values

Method 1: Fill with "Unknown"

```
df['director'] = df['director'].fillna("Unknown")
```

Method 2: Fill with Mode

```
df['country'] = df['country'].fillna(df['country'].mode()[0])
```

💡 **What does this do?**

- df['country'].mode() → finds the **most frequent value**
- [0] → picks the **top most frequent value**
- fillna(...) → replaces all **missing (NaN) values** with that value

Example

Before:

country
India
USA
India
NaN
UK
NaN

Step:

```
df['country'].mode()
```

Output: India

After:

country
India
USA
India
India
UK
India

🎯 **Why use this?**

Because:

✓ Keeps dataset size same

✓ Best for **categorical data** (country, gender, type)

✓ Uses most common value → logical assumption

⚠️ **When NOT to use?**

❌ If data is highly diverse

❌ If missing values are too many

❌ If mode doesn't make sense (e.g., salary)

Method 3: Drop rows (if very few)

```
df = df.dropna(subset=['date_added'])
```

💡 **What does this do?**

- Removes **entire rows** where date_added is missing

Example

Before:

title	date_added
Movie1	2020-01-01
Movie2	NaN
Movie3	2021-05-10

After:

title	date_added
Movie1	2020-01-01
Movie3	2021-05-10

👉 Row with missing value is **completely deleted**

🎯 **Why use this?**

✓ When missing rows are **very few**
✓ When column is **important (critical field)**
✓ When filling would create wrong data

⚠️ **When NOT to use?**

❌ If too many rows will be deleted
❌ If dataset becomes too small
❌ If losing important information

Interview Tip (Very Powerful Answer)

If interviewer asks: 👉 *"How do you handle missing values?"*

You can say:

"It depends on the context.
If the column is categorical, I may use mode to fill missing values.
If missing values are very few and the column is critical, I prefer dropping those rows to maintain data ***integrity***."

🚀 **Real-World Thinking (Important)**

In real jobs:

- **Country missing? → Fill**
- **Date missing? → Often drop**
- **Salary missing? → Maybe median**
- **Customer ID missing? → Drop (critical)**

PART 1: Real Interview Questions on Missing Values

Q1: How do you handle missing values in a dataset?

Strong Answer:

"It depends on the data type and business context.
For categorical variables, I use mode.
For numerical variables, I use mean or median depending on skewness.
If missing values are very few, I may drop them.
If a column has too many missing values (like >40%), I consider removing the column."

Q2: When would you drop missing values instead of filling them?

Answer:

"I drop missing values when:

- The number of missing rows is very small
- The column is critical (e.g., primary key, date)
- Imputing values may introduce bias or incorrect assumptions"

Q3: Mean vs Median — when to use?

Answer:

"Mean is used when data is normally distributed.
Median is preferred when data is skewed or has outliers because it is more robust."

Q4: What if 50% of data is missing?

Answer:

"If 50% of values are missing:

- I first check the importance of the column
- If it's not important → drop the column
- If it's important → use advanced imputation (group-based filling, predictive models)"

Q5: How do you detect missing values?

Answer:

df.isnull().sum()
Also mention: "I also check for hidden missing values like empty strings or 'NA', 'Unknown'."

Q6: What are hidden missing values?

Answer:

"Values like 'NA', 'N/A', 'Unknown', empty spaces — they are not treated as NaN but actually represent missing data."

🔥 PART 2: Case-Based Scenarios (VERY IMPORTANT)

These are exactly what companies like **Deloitte/KPMG** ask.

CASE 1: Customer Dataset

You have:

CustomerID	Age	Gender	Income
101	25	M	50000
102	NaN	F	60000
103	30	NaN	NaN

Q: How will you handle missing values?

Ans:

"I will handle each column differently:

- Age → Numerical → fill with median
- Gender → Categorical → fill with mode
- Income → Numerical → depends on distribution, usually median

I will not drop rows immediately because it reduces dataset size."

CASE 2: Date Column Missing

OrderID	OrderDate
1	2023-01-01
2	NaN

Q: What will you do?

Answer:

"OrderDate is a critical field.
I will:

- Try to retrieve missing data from source
- If not possible → drop the row
  because incorrect date imputation can affect time-based analysis."

CASE 3: 70% Missing Column

CustomerID	LoyaltyScore
1	NaN
2	NaN
3	80

Q: What will you do?

Answer:

"Since 70% of the data is missing:

- I will evaluate the importance of the column
- If not critical → drop it
- If important → consider advanced imputation techniques or external data"

CASE 4: Salary Column with Outliers

Salary
30000
35000
40000
1000000

Q: How will you fill missing values?

Answer: "Since data has extreme outliers, I will use median instead of mean because mean will be heavily influenced."

CASE 5: Duplicate + Missing

ID	Name	Email
1	John	NaN
1	John	NaN

Q: What will you do first?

Answer: "First, I will remove duplicates to avoid redundant processing.
Then I will handle missing values."

PART 3: Advanced Answers (To Impress Interviewer)

Smart Answer 1

"I don't apply one method globally.
I analyze missing data pattern — whether it's random or systematic."

Smart Answer 2

"I also consider business impact before removing or imputing data."

Smart Answer 3

"For time-series data, I may use forward fill or interpolation."

PART 4: Common Mistakes (Avoid These)

❌ Filling everything with mean
❌ Dropping too much data
❌ Ignoring missing values
❌ Not checking data distribution
❌ Not asking business context

PART 5: Practice Questions (Try Yourself)

Q1 If "City" column has missing values — what will you do?

Q2 If "CustomerID" is missing — what will you do?

Q3 If 5% data is missing randomly — what is best approach?

Step 3 — Fix Data Types

Check datatype:

```
df.dtypes
```

Example issue: *date_added → object (should be date)*

Fix it:

```
df['date_added'] = pd.to_datetime(df['date_added'])
```

OR

Problem = some times we get **extra space at the beginning like, " August 4, 2017" but it should be "August 4, 2017"**
↑ space here

Remove Extra Spaces

Fix:

```
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])
```

Step 4 — Remove Duplicates

Check duplicates:

```
df.duplicated().sum()
```

Remove them:

```
df = df.drop_duplicates()
```

Duplicates = **same rows repeated in dataset**

Example

Before:

ID	Name	Age
1	John	25
2	Alice	30
1	John	25

Check Duplicates

```
df.duplicated().sum()
```

💡 **What it does?**

- **df.duplicated()** → returns True/False for each row
- **.sum()** → counts how many duplicates exist

Example Output:

2

👉 Means 2 duplicate rows present

🔍 **Behind the scenes**

```
df.duplicated()
```

Output:

Row	Result
1	False
2	False
3	True

👉 Only repeated rows marked as **True**

Remove Duplicates

```
df = df.drop_duplicates()
```

💡 **What it does?**

- Removes **duplicate rows**
- Keeps only **first occurrence**

📊 **Example**

Before:

ID	Name	Age
1	John	25
2	Alice	30
1	John	25

After:

ID	Name	Age
1	John	25
2	Alice	30

Note:- np.int64(0), 👉 It simply means that *'There are ZERO duplicate rows in your dataset'*

Important Interview Concept

❓ *Should you always remove duplicates?*

👉 ❌ **NO**

🎯 **Correct Thinking:**

Situation	Action
Data entry error	Remove duplicates
Transaction data	Keep duplicates (valid)
Customer data	Remove duplicates

Interview Answer (Perfect)

If asked: 👉 *"How do you handle duplicates?"*

✅ **Answer:**

"I first identify duplicates using df.duplicated()
Then I analyze whether they are true duplicates or valid repeated records.
If they are redundant, I remove them using drop_duplicates().
If needed, I use subset to check duplicates based on specific columns."

🎯 **Summary:**

Step	Code
Check duplicates	`df.duplicated().sum()`
View duplicates	`df[df.duplicated()]`
Remove duplicates	`df.drop_duplicates()`

Imp Note

But WAIT ⚠️ (Important Real-World Insight)

Even if result = 0 → duplicates **might still exist logically**

Because:

❌ **Example (Hidden duplicates)**

ID	Name
1	John
1	john

👉 Not counted as duplicate (case difference)

🔥 **Check Properly (Advanced)**

1. Check duplicates based on ID

```
df.duplicated(subset=['show_id']).sum()
```

2. Convert text to lowercase first

```
df['title'] = df['title'].str.lower()
df.duplicated(subset=['title']).sum()
```

3. Remove spaces

```
df['title'] = df['title'].str.strip()
```

🧠 **Interview-Level Answer**

If interviewer asks:

> 👉 *"What if no duplicates are found?"*

✅ **Answer:** "Even if no exact duplicates are found, I still check for logical duplicates by standardizing text (lowercase, trimming spaces) and checking specific columns like IDs."

Step 5 — Fix Text Problems

Remove extra spaces:

```
df['title'] = df['title'].str.strip()
```

Convert to lowercase:

```
df['country'] = df['country'].str.lower()
```

Step 6 — Standardize Values

Example:

USA
United States
US

Fix them:

```
df['country'] = df['country'].replace({
   "united states": "USA",
   "us": "USA"
})
```

Step 7 — Create New Columns

Example:

Extract year from date.

```
df['year_added'] = df['date_added'].dt.year
```

Step 8 — Save Clean Dataset

```
df.to_csv("netflix_cleaned.csv", index=False)
```

Step 9 — Verify Data

Always check again:

```
df.info()
df.isnull().sum()
df.duplicated().sum()
```

Everything should be clean.